企业内部自然语言数据查询系统

完整流程方案文档（RAG + Workflow Agent + 前后端 + 数据库 + Ollama）

适用场景	将现有 xlsx 数据导入数据库，在前端 AI 对话界面中用自然语言查询业务数据，并由本地部署的大模型理解需求、对齐字段语义、生成可控查询计划并返回结果。
核心结论	建议采用“RAG 增强的 NL2DSL / NL2Query + 安全执行”的架构，而不是让模型直接写 SQL。Agent 建议采用轻编排型，而不是一开始上全自动多工具智能体。
方案定位	这不是一个“自由发挥型聊天机器人”，而是一个“受控的自然语言查数系统”。RAG 负责理解业务与字段语义，Workflow Agent 负责编排步骤，后端代码负责安全执行。

文档目标：给出从数据入库、元数据建设、RAG 设计、Agent 编排、后端执行、前端对话、日志治理到迭代路线的完整落地方案。

1. 项目目标与设计原则

项目目标

- 把现有 xlsx 数据稳定地导入数据库，并形成可持续更新的数据底座。
- 让业务人员通过自然语言提出复杂、模糊甚至带业务口径的查数需求。
- 由本地部署 Ollama 模型理解需求，并结合 RAG 检索到的业务语义、字段定义、指标口径和样例查询，生成结构化查询计划。
- 后端对计划进行严格校验，再由代码生成 SQL / ORM 查询并执行，最终以表格、图表和文字总结返回前端。

设计原则

- 模型负责理解，代码负责执行。
- RAG 负责把“用户说法”对齐到“数据库真实语义”。
- Agent 只负责编排流程，不负责绕过约束。
- 任何可执行动作都必须通过白名单、类型校验和权限控制。
- 先做最小闭环，再扩展多表、图表、多轮上下文和复杂分析能力。

> **架构判断** 你这个系统确实需要 RAG，但 RAG 的对象主要是元数据、字段语义、指标口径和示例查询，而不是把整个数据库原样塞进 prompt。

2. 整体架构概览

推荐采用如下六层结构：

层级	职责	核心输入	核心输出
数据层	xlsx 导入数据库；维护表结构、元数据、样例值、字典映射。	Excel 文件、字段配置	数据库表 + metadata
检索层	按用户问题检索相关字段、指标口径、示例查询和历史模板。	用户问题、metadata 索引	RAG 上下文包
规划层	调用 Ollama 生成 DSL / 查询计划，而不是直接生成 SQL。	用户问题、RAG 上下文	结构化查询计划
校验执行层	校验任务类型、字段、	DSL	查询结果集

	聚合、权限和时间范围，再安全执行查询。		
表达层	对结果做自然语言总结、图表配置建议和异常提示。	结果集、业务口径	文字总结/图表配置
交互层	聊天界面、表格、图表、历史记录、调试信息。	后端响应	可视化页面

```
用户自然语言问题
    -> RAG 检索相关 schema / 字段语义 / 指标口径 / 示例
    -> Ollama 生成 DSL
    -> 后端校验 DSL
    -> 代生成 SQL / ORM 查询
    -> 执行数据库查询
    -> LLM 对结果总结
    -> 前端表格 + 图表 + 说明返回
```

3. 端到端业务流程

推荐把一次查询请求拆成 9 个明确步骤：

1. 前端发送用户问题、用户身份、会话上下文和可选筛选条件。
2. 后端做基础预处理：清理文本、抽取明显时间表达、识别是否需要追问。
3. RAG 检索最相关的表、字段、业务解释、示例值、指标口径和类似查询模板。
4. 将“用户问题 + RAG 上下文 + DSL 约束 + few-shot 示例”发送给 Ollama。
5. 模型输出结构化 DSL，后端解析为 Pydantic / JSON Schema 对象。
6. 后端执行校验：任务类型、字段白名单、聚合方式、过滤条件、权限范围、时间字段可用性。
7. 校验通过后由代码生成 SQL 或 ORM 查询；如不通过，返回可解释错误。
8. 执行查询，必要时做分页、缓存、结果裁剪、衍生指标计算。
9. 将结果集交给表达层：生成业务总结、图表建议、口径说明和异常提醒。

建议 即使模型足够强，也不要让它直接写 SQL 并执行。让模型生成结构化 DSL，再由后端代码执行，是稳定性和可治理性的关键。

4. RAG 设计：让业务复杂表达与数据库语义对齐

在你的场景里，RAG 的目的不是回答开放式知识问答，而是把“业务说法”映射成“数据层真实含义”。

4.1 RAG 素材池

- 表结构说明：表名、字段名、字段类型、主键、外键、更新时间。
- 字段业务语义：字段中文解释、别名、缩写、常见误写、示例值。
- 指标口径文档：例如“活跃用户”“高价值客户”“新增用户”的定义。
- 时间口径说明：按注册时间、按最后活跃时间、按订单创建时间等。
- 示例查询：自然语言问题、对应 DSL、对应结果解释。
- 业务规则：哪些字段允许过滤、哪些字段允许分组、哪些字段只可展示不可暴露。

4.2 推荐的 metadata 字段

字段	说明	示例	用途
field_name	数据库字段名	customer_type	执行查询的真实映射
aliases	别名、业务口语、常见缩写	客户类型 / 客群 / 客户分层	支持用户非标准表述
description	字段业务解释	区分个人、企业等客群属性	帮助模型做语义对齐
examples	示例值	个人, 企业	帮助模型推断过滤值
allowed_ops	允许的操作	filter, groupby	做安全校验
sensitivity	敏感级别	public / internal / restricted	做权限控制

4.3 检索策略

- 第一层召回：按关键词、别名、字段名、表名做高召回检索。
- 第二层重排：根据用户问题中的实体、时间、指标和动作词做相关性排序。
- 第三层打包：把最相关的字段解释、指标口径和 2-4 个示例查询组合成上下文包。
- 当表较多时可引入向量库；当规模还小，可先用关键词/BM25 方案。

```
RAG Context Package
- Relevant tables: users, orders
- Relevant fields: register_time, platform, customer_type
- Metric definition: "high_value_user" = ...
- Allowed group by: platform, customer_type
- Example query #1 -> DSL
```

```
- Example query #2 -> DSL
```

5. Workflow Agent 设计：为什么要轻 Agent

建议采用 Workflow Agent，而不是一开始使用全自动自由规划的通用 Agent。

5.1 为什么不用重 Agent

- 自由 Agent 容易越权调用工具、重复试错、不可控地拼 SQL。
- 你的场景目标清晰，属于典型受限任务，固定流程比开放探索更稳定。
- 真正复杂的是语义对齐和口径判断，不是工具数量。
- 审计与复现要求高时，确定性流程远优于自由规划。

5.2 轻 Agent 的职责

- Tool 1: 检索 metadata / 指标口径 / 示例。
- Tool 2: 生成 DSL。
- Tool 3: 校验 DSL。
- Tool 4: 执行查询。
- Tool 5: 总结结果。

> **推荐表达** 你可以对外说系统使用了 Agent，但实现上更准确的说法是“带有 RAG 的工作流编排系统（workflow-based agentic pipeline）”。

6. 数据层设计：xlsx 入库、元数据与指标口径

6.1 xlsx 入库策略

- 保留原始 xlsx 作为审计输入，不直接覆盖。
- 将清洗后的数据写入 SQLite 作为第一阶段数据库；后续可迁移到 PostgreSQL。
- 建立导入日志：导入时间、文件名、行数、异常值、字段映射、版本号。
- 如果 Excel 结构不稳定，增加字段映射配置文件，显式声明列名到数据库字段的映射。

6.2 数据库建议

- MVP 阶段：SQLite 足够。
- 多用户、多并发、权限复杂后切 PostgreSQL。
- 优先把事实表和维度表拆清楚，避免所有字段都堆在一张表里。

6.3 数据字典与指标口径库

- 数据字典解决“字段是什么”。
- 指标口径库解决“业务概念怎么计算”。
- 示例查询库解决“用户常见表达如何映射”。

7. 后端服务设计

推荐使用 FastAPI 作为后端主框架。建议模块化拆分如下：

模块	主要职责	备注
database.py	数据库连接、建表、xlsx 导入、基础查询	可先用 sqlite3 / SQLAlchemy
metadata.py	字段语义、指标口径、示例查询、RAG 检索	支持关键词与向量检索双模式
planner.py	调用 Ollama，输出 DSL	严格要求 JSON 输出
validator.py	白名单校验、权限校验、类型校验	查询前的硬门禁
executor.py	DSL -> SQL / ORM -> 执行结果	不可让 LLM 直接越过此层
answerer.py	结果总结、图表建议、口径说明	可单独使用较小模型
main.py	API 入口、路由、鉴权、日志	统一请求追踪

```
POST /chat/query
POST /import/xlsx
GET  /schema
GET  /history
GET  /healthz
```

8. 前端对话与结果展示设计

前端建议采用 React + Vite。第一版界面要追求“查数效率”而不是炫技。

- 聊天区：展示用户问题、系统结果、异常提示。
- 结果区：表格、图表、口径说明。
- 调试区：可选显示 DSL、命中字段、执行耗时。

- 历史区：最近查询、常用问题模板、收藏查询。
- 权限提示：如果用户无权查看某字段，前端要接收并展示后端返回的受限说明。

> **交互建议** 对于模糊问题，系统可以给出一到两个澄清选项，例如“你说的新增用户，是按注册时间还是按首单时间？” 但不要每次都追问，能从口径库判断时就直接执行。

9. Prompt、DSL 与查询执行规范

9.1 DSL 的作用

- 把自然语言理解结果变成结构化、可校验、可执行的中间层。
- 让 LLM 与数据库之间多一层安全隔离。
- 方便日志审计、缓存和测试。

9.2 示例 DSL

```
{
  "task": "distribution_summary",
  "subject": "users",
  "time_range": "30d",
  "date_field": "register_time",
  "filters": {"customer_type": "企业"},
  "group_by": ["platform"],
  "metrics": ["count"],
  "sort": [{"field": "count", "order": "desc"}],
  "limit": 20
}
```

9.3 Prompt 组成

- 系统指令：只能输出合法 JSON，不要解释。
- 能力边界：只能使用给定字段、任务和时间口径。
- RAG 上下文：字段说明、口径定义、示例值、few-shot 示例。
- 用户问题：原始自然语言。

9.4 校验清单

- task 是否属于允许集合。
- subject、group_by、filter 字段是否存在且允许。
- 日期字段是否可用于 time_range。
- 聚合方式是否在白名单内。
- 用户是否有权限访问目标字段与结果维度。
- limit、sort、分页是否安全。

10. 安全、治理与可观测性

为了让系统长期可维护，建议一开始就补上治理能力。

- 请求日志：原问题、RAG 命中内容、DSL、SQL、耗时、结果摘要。
- 错误分层：模型理解错误、口径不明确、字段不存在、权限拒绝、数据库异常。
- 测试集：沉淀 50-100 条高频业务问题，作为回归集。
- 缓存：对高频、低时效性问题缓存 DSL 或查询结果。
- 权限：区分公共字段、内部字段、敏感字段。
- 脱敏：用户级数据、手机号、邮箱等敏感信息必须做展示控制。

> **可观测性关键** 你后面调系统，最有价值的不是“模型参数”，而是每次请求的可追踪日志：用户怎么问、RAG 命中了什么、模型怎么规划、最后为什么执行成功或失败。

11. 实施路线图

推荐按四个阶段推进，每个阶段都要形成可演示版本。

段	目标	产出	风险控制点
阶段 1	打通单表闭环	xlsx 入库 + metadata + DSL + 查询接口	不做自由 SQL
阶段 2	接入前端对话页	聊天界面 + 表格展示 + 错误提示	保留调试面板
阶段 3	引入 RAG 与口径库	字段语义检索 + 指标口径 + 示例查询库	控制上下文长度
阶段 4	扩展多表与复杂分析	join、图表、历史模板、多轮上下文	避免性能和权限失控

12. 推荐 MVP 范围

建议你的第一版只覆盖下面这些范围：

- 1 个或 2 个核心 xlsx 数据源。
- 1 套清晰的数据字典和字段别名映射。
- 4 类左右的固定任务：列表、单值统计、分组分布、趋势统计。
- 1 个 Ollama 模型用于 DSL 规划，1 个模型或同一模型用于结果总结。
- 1 个聊天页面 + 1 个表格区 + 1 个调试区。

- 完整日志和最小权限控制。

一句话落地建议 先把它做成“RAG 增强的自然语言查数系统”，等这个闭环稳定后，再增加更强的 Agent 行为，而不是反过来。

附：推荐项目目录

```
project/
  backend/
    main.py
    database.py
    metadata.py
    planner.py
    validator.py
    executor.py
    answerer.py
    prompts/
  frontend/
    src/
      components/
  data/
    raw/
    metadata/
    app.db
```